

FUNCTIONS

Functions are a great tool to help you organize your code, but to write your own functions, you must understand `def` statements, parameters, arguments, and return values. Functions also bring to light new programming concepts, such as the call stack and scopes.

The following activities test your ability to create functions and use them effectively. Almost every program you write of significant length is better served by including functions, so it's important to understand their behavior.



LEARNING OBJECTIVES

- Master the structure of a `def` statement and how it can include parameters.
- Identify the value a function returns and know how to set its return value with the `return` keyword.
- Understand the behavior of the `None` value and when functions return it.
- Be able to explain how Python represents function calls using the underlying call stack.
- Understand the concepts of global and local scope and be able to identify a variable's scope.
- Know how to handle exceptions using `try` and `except`.
- Gain the ability to write your own functions for a variety of use cases.



Practice Questions

Functions are the primary way to compartmentalize your code into logical groups. Answer the following questions to test your ability to work with the components of functions.

Creating Functions

The first line of any function definition is a `def` statement. If a function can accept arguments, this `def` statement contains parameters, which are variables that store arguments. For each of the following, answer “yes” if it is a valid Python `def` statement; answer “no” if it is an invalid Python `def` statement.

1. `def hello:`
2. `define hello(name):`
3. `def h(name):`

```
4. hello(name):  
  
5. def:  
  
6. def hello():  
  
7. def hello(name):
```

Arguments and Parameters

The following questions further test your ability to recognize the elements of function definitions.

8. How can you tell that the following code defines a function rather than calls a function?

```
def say_hello():
```

9. What are the parameters in the following `def` statement?

```
def add_club_member(first_name, last_name):
```

10. In the following code, is 'Albert' a parameter or an argument?

```
say_hello('Albert')
```

The code in the block that follows the `def` statement is the body of the function. To correctly understand a program, you must be able to distinguish the body, which runs only when the function is called, from the code that exists outside the function. To that end, each of the following examples is a complete program; describe what it prints.

11.

```
def say_hello():  
    print('Hello')
```

12.

```
def say_hello():  
    print('Hello')  
for i in range(3):  
    say_hello()
```

13.

```
def say_hello():  
    for i in range(3):  
        print('Hello')  
say_hello()  
say_hello()
```

Return Values and return Statements

In general, the value to which a function call evaluates is called the *return value* of the function, but you can also specify the return value with a `return` statement, which consists of the following:

- The `return` keyword
- The value or expression that the function should return

To test whether you understand the data types returned by Python functions, answer the following questions about `return` statements.

14. What is the data type of the return value in the following function?

```
def enter_password(password):  
    if password == 'swordfish':  
        return True  
    else:  
        return False
```

15. In the previous `enter_password()` function, what can the data type of the `password` parameter be?

16. What is the data type of the return value in the following function?

```
def get_greeting():  
    print('What is your name?')  
    name = input()  
    return 'Hello, ' + name
```

The None Value

In Python, a value called `None` represents the absence of a value. Behind the scenes, Python adds `return None` to the end of any function definition with no `return` statement.

It's important to understand how `None` works so that you can know what your functions are returning. Determine whether the following expressions involving `None` evaluate to `True` or `False`. (You can enter the expression into the interactive shell to find out.)

17. `None == True`

18. `None == False`

19. `None == 'None'`

20. `None == None`

21. `None == 'hello'`

22. `None == 0`

23. `None == -1.5`

The Call Stack

The *call stack* is how Python remembers where to return the execution after each function call. The call stack isn't stored in a variable in your program; rather, it's a section of your computer's memory that Python handles automatically behind the scenes. When your program calls a function, Python creates a *frame object* on the top of the call stack. Frame objects store the line number of the original function call so that Python can remember where to return. Answer the following questions about the frame objects, the call stack, and function calls.

24. What does a stack frame object represent?

25. When is a stack frame object pushed to the top of the call stack?

26. When is a stack frame object popped off the top of the call stack?

27. What does the stack frame object at the top of the call stack represent?

28. A call to a function named `spam()` is made. Then, a call to an `eggs()` function is made. Next, `eggs()` returns. After that, a call to a `bacon()` function is made. What does the call stack look like at this point?
29. A program has absolutely no function calls in it. What does the call stack look like while the program runs?

Local and Global Scopes

Only code within a called function can access the parameters and variables assigned in that function. These variables are said to exist in that function's *local scope*. By contrast, code anywhere in a program can access variables that are assigned outside all functions. These variables are said to exist in the *global scope*. Answer the following questions about global variables, local variables, and scopes.

30. Are function parameters global variables or local variables?
31. A variable in a function is marked with the `global` statement. Is it a global or local variable?
32. Can a variable be both global and local?
33. If a global `spam` variable exists, and a function has a `spam = 42` assignment statement and no `global spam` statement, is the `spam` variable in the function local or global?
34. If a global `spam` variable exists, and a function has a `spam = 42` assignment statement as well as a `global spam` statement, is the `spam` variable in the function local or global?
35. If a global `spam` variable exists, and a function never assigns `spam` a value and has no `global spam` statement, the function uses the `spam` variable (such as in `print(spam)`). Is the `spam` variable in the function local or global?

Many errors occur because programmers mistakenly identify the scope in which a variable exists. To test whether you correctly understand Python's scoping rules, determine what each of the following programs outputs.

36.

```
def func(spam):
    print(spam)
spam = 'dog'
func('cat')
```

37.

```
def func(eggs):
    print(spam)
spam = 'dog'
func('cat')
```

38.

```
def func():
    spam = 'cat'
spam = 'dog'
func()
print(spam)
```

39.

```
def func():
    global spam
    spam = 'cat'
spam = 'dog'
func()
print(spam)
```

40.

```
def func():
    global spam
    print(spam)
    spam = 'cat'
spam = 'dog'
func()
```

41.

```
def func():
    print(spam)
    spam = 'cat'
spam = 'dog'
func()
```

Exception Handling

Usually, getting an error, or *exception*, in your Python program means the entire program will crash. But programs can also handle errors with `try` and `except` statements. The code that could potentially have an error is put in a `try` clause. The program execution moves to the start of the following `except` clause if an error happens. For each of the following programs, determine whether the program would crash if the user entered a non-number.

42.

```
print('Enter a number:')
number = int(input())
try:
    print('You entered a number.')
except:
    print('You did not enter a number.')
```

43.

```
print('Enter a number:')
try:
    number = int(input())
    print('You entered a number.')
except ValueError:
    print('You did not enter a number.')
```

44.

```
print('Enter a number:')
try:
    number = int(input())
    print('You entered a number.')
```

```
except ZeroDivisionError:
    print('You did not enter a number.')
```



Practice Projects

Now you'll create some functions to practice what you've learned.

Transaction Tracker

Write a function named `after_transaction()` that returns the amount of money in an account after a transaction. The two parameters for this function are `balance` and `transaction`. They will both have integer arguments. The `balance` is how much money is currently in the account, and the `transaction` is how much to add or remove from the account (based on whether `transaction` is a positive or negative integer).

This operation is more complicated than just `return balance + transaction`. If the `transaction` is negative and would overdraw the account (that is, if `balance + transaction` is less than zero), then the `transaction` should be ignored and the original `balance` returned. For example, calling the function from the interactive shell should look like this:

```
>>> after_transaction(500, 20)
520
>>> after_transaction(300, -200)
100
>>> after_transaction(3, -1000)
3
>>> after_transaction(3, -4)
3
>>> after_transaction(3, -3)
0
```

Arithmetic Functions Without Arithmetic Operators

Let's create `add(number1, number2)` and `multiply(number1, number2)` functions that add and multiply their arguments without using the `+` or `*` operators. These functions will be quite inefficient, but don't worry; the computer doesn't mind.

Imagine that we start with this `plus_one()` function, which is the only function where we'll allow the use of the `+` operator:

```
def plus_one(number):
    return number + 1
```

For example, calling `plus_one(5)` returns 6 and calling `plus_one(6)` returns 7.

Your `add()` function should not use the `+` operator; rather, it should have loops that repeatedly call the `plus_one()` function to perform the addition operation on the operands passed as parameters. After all, the operation `4 + 3` is the same as `4 + 1 + 1 + 1`. Your `add()` function is expected to handle positive integers only.

If you need a hint, start with the following template:

```
def add(number1, number2):
    total_sum = ____
```

```
for i in range(number2):
    ____ = plus_one(____)
return ____
```

Your `multiply()` function should work in the same way: Avoid using the `*` operator, and instead use a loop to repeatedly call your `add()` function. After all, the operation `3 * 5` is the same as `3 + 3 + 3 + 3 + 3` or `5 + 5 + 5`.

It's a good idea to make sure your `add()` function works before beginning on `multiply()`. Also note that `2 + 8` is the same as `8 + 2` and `2 * 8` is the same as `8 * 2`.

Save these functions in a file named *arithmeticFunctions.py*.

Tick Tock

The `time.sleep()` function, which pauses program execution for a specified amount of time, is useful, but rather plain. Let's write our own `tick_tock(seconds)` function that also pauses for `seconds` amount of time but prints `Tick...` and `Tock...` each second while waiting.

For example, calling the function from the interactive shell should look like this (with a one-second pause after each line of output):

```
>>> tick_tock(4)
Tick...
Tock...
Tick...
Tock...
>>> tick_tock(3)
Tick...
Tock...
Tick...
```

You may assume that the `seconds` parameter always has a positive integer argument. Keep in mind that if the argument for `seconds` is odd, the last thing the function should print is `Tick...`

Save this `tick_tock()` function in a file named *tickTockPrint.py*.